

AgriPrice: An End-to-End MLOps Pipeline for Kenyan Agricultural Commodity Price Forecasting

Cynthia Gichuki and Gauri Nair

School of Computer Science and Information Technology

Lucerne University of Applied Sciences and Arts

Rotkreuz, Switzerland

cynthia.gichuki@stud.hslu.ch, gauri.nair@stud.hslu.ch

<https://github.com/CynthiaGichuki/AgriPrice>

Abstract—Agricultural commodity price fluctuation presents a persistent challenge for farmers, traders, and policymakers across sub-Saharan Africa. In Kenya, unpredictable price fluctuations for staple crops such as Maize, Tomatoes, and Wheat can have severe economic consequences for market participants who lack access to reliable forecasting tools. This paper presents AgriPrice, which is an end-to-end machine learning operations (MLOps) pipeline that predicts both wholesale and retail commodity prices in Kenyan Shillings (KES). The system employs a chained XGBoost architecture in which a wholesale price model executes first and its prediction is passed as an input feature to a downstream retail price model, reflecting the real-world causal dependency between wholesale and retail markets. Input features consist of categorical market identifiers with temporal signals derived from the query date. The trained models are served through a FastAPI prediction service containerised with Docker and deployed automatically via a GitHub Actions CI/CD pipeline. Experiment tracking is handled through MLflow, which records model parameters and performance metrics across training runs. A dedicated monitoring pipeline tracks data drift and model performance over time, with results stored in a SQL database and visualised through Grafana dashboards. Every code change triggers seventeen automated tests covering model inference logic and HTTP API contract compliance before a Docker image is built and pushed to the GitHub Container Registry. The result is a reproducible, testable, and deployable ML system that demonstrates the full MLOps lifecycle from model training and serving to monitoring and continuous automated delivery.

Index Terms—MLOps, Price Forecasting, XGBoost, Chained Models, FastAPI, Docker, CI/CD, GitHub Actions, MLflow, Grafana, Kenya Agricultural Markets.

I. INTRODUCTION

Kenya's agricultural sector accounts for approximately 33 percent of gross domestic product and employs over 40 percent of the total workforce [1]. Despite its economic centrality, the sector is characterised by significant price volatility driven by seasonal production cycles, regional supply imbalances, post-harvest losses, and currency fluctuations. Farmers selling produce at market and traders purchasing inventory for resale lack access to systematic price forecasting tools, leaving them dependent on informal networks and historical intuition when making high-stakes economic decisions.

Machine learning offers a promising avenue for commodity price forecasting given the availability of historical market data published by institutions such as the Kenya National Bureau

of Statistics (KNBS) and the Food and Agriculture Organisation (FAO). Gradient-boosted tree models in particular have demonstrated strong performance on structured tabular data with mixed categorical and numerical features, making them well-suited to this domain.

However, developing a model that achieves reasonable predictive accuracy represents only the first step towards a useful system. A model that exists only as a trained artifact on a developer's laptop cannot deliver value to farmers or traders. The engineering work required to make a model reliably available, testable, and automatically deployable is the domain of Machine Learning Operations (MLOps) [2].

The AgriPrice project was developed as part of the ARTIFIN course at the Hochschule Luzern (HSLU), the pipeline is largely divided into six primary components. First, selecting a publicly available dataset and defining a clear prediction goal. Second, training machine learning models with full experiment tracking. Third, deploying those models as a reliable API service. Fourth, monitoring the performance of the deployed models over time. Fifth, visualising monitoring results through dashboards. Sixth, automating the testing and deployment process through a CI/CD pipeline (according to Fig 1).

This paper documents the full system: the data and machine learning task in Section III, the modelling and reproducibility approach in Section IV, experiment tracking in Section V, the deployment architecture in Section VI, the monitoring pipeline in Section VII, visualisation and dashboards in Section VIII, the CI/CD automation in Section IX, the testing strategy in Section X, and a discussion of limitations and future work in Section XI.

II. RELATED WORK

A. Agricultural Price Forecasting

Prior work on agricultural commodity price forecasting has explored a range of model architectures. Early approaches relied on autoregressive integrated moving average (ARIMA) models, which are well-suited to univariate time series but struggle to incorporate the multi-dimensional feature context that characterises commodity pricing [3]. More recent work has applied gradient-boosted trees and neural networks, which

naturally handle mixed feature types and non-linear interactions. Darekar and Reddy [4] demonstrated the utility of XGBoost for forecasting vegetable prices in India, achieving competitive performance against ARIMA baselines. The chained or cascaded modelling approach adopted in AgriPrice builds on this literature by encoding the causal relationship between wholesale and retail markets explicitly.

B. XGBoost for Tabular Data

XGBoost (Extreme Gradient Boosting) was introduced by Chen and Guestrin [5] as a scalable tree boosting system with built-in regularisation, efficient handling of missing values, and parallelised training. It has consistently demonstrated state-of-the-art performance on structured tabular datasets across academic benchmarks and Kaggle competitions. Its advantages for commodity price forecasting include interpretable feature importance scores and the ability to handle categorical features after ordinal or target encoding without requiring the sparse one-hot encoded matrices typical of deep learning pipelines.

C. MLOps and the Serving Pipeline

MLOps refers to the set of practices that apply DevOps principles to machine learning system development [6]. Kreuzberger et al. [2] provide a systematic overview of MLOps architectures, distinguishing three primary pipelines (according to Fig 1): the training pipeline, the serving pipeline, and the monitoring pipeline. AgriPrice addresses all three pipelines: the training pipeline produces model artifacts from historical market data, the serving pipeline delivers predictions to end users through a FastAPI API, and the monitoring pipeline continuously checks whether the deployed models are still performing well over time. Continuous integration and continuous delivery practices are applied to the serving pipeline to automate quality gates and deployment.

D. FastAPI and Containerised ML Services

FastAPI has emerged as the preferred framework for ML serving APIs in Python due to its automatic request validation through Pydantic schemas, auto-generated interactive documentation, and high throughput based on the ASGI standard [7]. Docker containerisation packages the service and its dependencies into a portable image that executes identically across development, CI, and production environments [8], eliminating the class of environment-specific failures that commonly affect ML deployments.

III. DATA AND MACHINE LEARNING TASK

A. Prediction Task

The primary machine learning task is supervised regression: given a set of features describing a commodity, its market context, and the date of the query, predict the expected price in Kenyan Shillings. AgriPrice produces two distinct predictions for each request: a wholesale price estimate and a retail price forecast. The two predictions are returned together in a single API response as `stage_1_wholesale_estimate` and `stage_2_retail_forecast` (Fig 2).

B. Input Feature Set

The feature set is divided into two categories as summarised in Table I. Categorical features describe the identity and market context of the commodity being queried. Temporal features capture the time dimension and are derived programmatically from the input date string at inference time.

TABLE I
INPUT FEATURE SET FOR AGRIPRICE MODELS

Feature	Type	Description
Commodity	Categorical	Crop name (e.g. Maize, Tomatoes)
Classification	Categorical	Crop type (e.g. Grain, Vegetable)
Market	Categorical	Market location name
County	Categorical	Administrative county
Unit	Categorical	Unit of measure (e.g. KG)
Month	Temporal	Month of the query date
Year	Temporal	Year of the query date
Day of Week	Temporal	Day of week (0–6)
Week of Year	Temporal	ISO week number (1–53)

The categorical features are encoded using scikit-learn’s `OrdinalEncoder` with the `handle_unknown` parameter set to `use_encoded_value` and an unknown value of `-1`. This configuration ensures that commodity names, markets, counties, or units not seen during training are assigned a consistent fallback encoding at inference time rather than raising an exception.

C. Target Variable and Transformation

The target variable for both models is commodity price in Kenyan Shillings. Raw price distributions are right-skewed due to the presence of high-priced commodities measured in different units. A logarithmic transformation is applied to the target before training using the `numpy` function `log1p` (natural logarithm of one plus the price) and inverted at inference time using `expm1`. This transformation stabilises the target variance and improves gradient convergence during XGBoost training.

D. Chained Model Justification

The decision to use a chained architecture rather than a single multi-output model reflects the economic structure of Kenyan agricultural markets. Wholesale prices are determined first at origin markets and supply depots; goods are then transported and sold at retail markets where prices incorporate transport costs, handling fees, and retailer margins. This structural dependency means that the retail price has a causal relationship with the wholesale price that is not captured by treating the two targets as independent outputs. By conditioning the retail prediction explicitly on the wholesale estimate, the chained model encodes real-world market structure directly into the inference logic.

IV. MODELLING AND REPRODUCIBILITY

A. Chained XGBoost Architecture

The modelling pipeline consists of two XGBoost regressors [5] operating in sequence. The Stage 1 model (wholesale) receives a nine-dimensional feature vector comprising five

AgriPrice — End-to-End ML Pipeline

Kenyan Agricultural Price Forecasting System

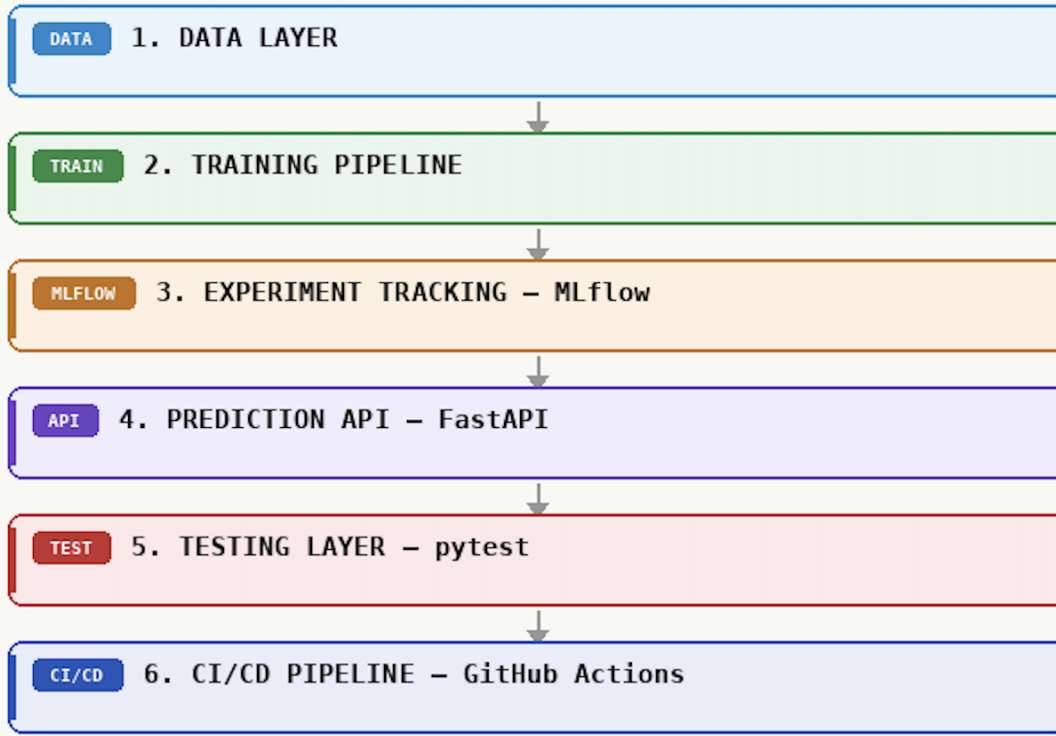


Fig. 1. AgriPrice full MLOps pipeline overview

1	Date	Commodity	Classification	Market	Unit	Wholesale	Retail	County	Month	Year	DayOfWeek	WeekOfYear		
2015	2023-11-09	Beans	Rosecco	Standard	Kwange	Kg	150.0	175.0	Nairobi	11	2023	3,45		
2016	2023-11-09	Beans	Rosecco	Standard	Nyamira	Kg	153.33	188.0	Nairobi	11	2023	3,45		
2017	2023-11-09	Beans	Rosecco	Standard	Gikoma	Kg	146.0	226.0	Nairobi	11	2023	3,45		
2018	2023-09-09	Beans	Rosecco	Standard	Gikoma	Kg	150.0	220.0	Nairobi	9	2023	5,30		
2019	2023-09-09	Beans	Rosecco	Standard	Malindi	Market	Kg	150.0	180.0	Nakuru	9	2023	5,30	
2020	2023-08-09	Beans	Rosecco	Standard	Makindu	Kirinyaga	Kg	111.11	120.0	Kirinyaga	8	2023	2,32	
2021	2023-08-09	Beans	Rosecco	Standard	Makindu	West Pokot	Kg	132.0	160.0	West Pokot	8	2023	2,32	
2022	2023-08-09	Beans	Rosecco	Standard	Makindu	West Pokot	Kg	111.11	130.0	Kirinyaga	8	2023	2,32	
2023	2023-08-09	Beans	Rosecco	Standard	Kisumu	Harijani	Market	Kg	150.0	270.0	Frans-Murua	8	2023	2,32
2024	2023-08-09	Beans	Rosecco	Standard	Chuka	Kg	133.33	180.0	Tharaka-Nithi	8	2023	2,32		
2025	2023-08-09	Beans	Rosecco	Standard	Gikoma	Kg	150.0	220.0	Nairobi	8	2023	2,32		
2026	2023-08-09	Beans	Rosecco	Standard	Lumbi	Kg	166.67	170.0	Wajir	8	2023	2,32		
2027	2023-07-09	Beans	Rosecco	Standard	Makindu	Kg	160.0	180.0	Mogadi	7	2023	6,27		
2028	2023-07-09	Beans	Rosecco	Standard	Kinili	town	Kg	175.0	200.0	Rungu	7	2023	6,27	
2029	2023-07-09	Beans	Rosecco	Standard	Beans	Market	Kg	164.44	220.0	Musia	7	2023	6,27	
2030	2026-03-19	Cabbages	Standard	Musini	Kg	23.11	Kakamega	3	2026	4,12				
2031	2026-03-19	Cabbages	Standard	Kutus	Kg	10.0	20.0	Kirinyaga	3	2026	4,12			
2032	2026-03-19	Cabbages	Standard	Kerugoya	Kg	15.0	20.0	Kirinyaga	3	2026	4,12			
2033	2026-03-19	Cabbages	Standard	Kinanga	Kg	25.0	33.33	Kwale	3	2026	4,12			
2034	2026-03-18	Cabbages	Standard	Mulot	Kg	10.0	15.0	Bomet	3	2026	2,12			
2035	2026-03-18	Cabbages	Standard	Musa	Kg	15.0	25.0	Meru	3	2026	2,12			
2036	2026-03-18	Cabbages	Standard	Kangeta	Kg	15.0	20.0	Meru	3	2026	2,12			
2037	2026-03-18	Cabbages	Standard	Diani	Market	Kg	80.0	180.0	Kwale	3	2026	2,12		
2038	2026-03-18	Cabbages	Standard	Kakamega	Town	Kg	20.0	40.0	Kakamega	3	2026	2,12		
2039	2026-03-17	Cabbages	Standard	Hulu	Kg	80.0	Kisumu	3	2026	1,12				
2040	2026-03-17	Cabbages	Standard	Kerugoya	Kg	15.0	20.0	Kirinyaga	3	2026	1,12			
2041	2026-03-17	Cabbages	Standard	Hogonyiek	Market	Kg	10.0	13.33	Bomet	3	2026	1,12		
2042	2026-03-17	Cabbages	Standard	Meru	Kg	50.0	60.0	Kisumu	3	2026	1,12			
2043	2026-03-17	Cabbages	Standard	Mukhi	Kg	4.0	30.0	Meru	3	2026	1,12			
2044	2026-03-17	Cabbages	Standard	Kapken	Kg	15.0	20.0	Bomet	3	2026	1,12			
2045	2026-03-16	Cabbages	Standard	Chimbo	Kg	10.0	20.0	Bomet	3	2026	0,12			
2046	2026-03-16	Cabbages	Standard	Kutus	Kg	15.0	25.0	Kirinyaga	3	2026	0,12			
2047	2026-03-16	Cabbages	Standard	Kajulu	Kg	10.0	15.0	Kirinyaga	3	2026	0,12			
2048	2026-03-15	Cabbages	Standard	Kilgori	Kg	30.0	Kisumu	3	2026	6,11				
2049	2026-03-14	Cabbages	Standard	Kakamega	Town	Kg	20.0	33.33	Kakamega	3	2026	5,11		
2050	2026-03-14	Cabbages	Standard	Kahati	Market	Kg	33.33	33.33	Muranga	3	2026	5,11		
2051	2026-03-13	Cabbages	Standard	Kerugoya	Kg	15.0	20.0	Kirinyaga	3	2026	4,11			
2052	2026-03-13	Cabbages	Standard	Musini	Kg	27.78	Kakamega	3	2026	4,11				

Fig. 2. Dataset

ordinal-encoded categorical features and four temporal features. It produces a single predicted wholesale price after inverse log transformation. This prediction is then appended to the original feature vector as a tenth feature for the Stage 2 model (retail), which produces the final retail price forecast.

At inference time the FastAPI service enforces the execution order internally. The caller submits a single JSON request

body, the service executes Stage 1, appends the result, executes Stage 2, and returns both predictions. The caller is not required to manage the intermediate state. This design isolates the chaining logic within the service boundary and prevents misuse of the models in isolation when the chained context is required (Fig 3).

B. Feature Engineering

The full feature engineering pipeline involves two transformations applied consistently during both training and inference to ensure reproducibility.

Ordinal encoding is applied to all five categorical features using separate `OrdinalEncoder` instances for the wholesale and retail models. Using separate encoders is necessary because the retail encoder must be fit on the same training data as the retail model. The encoders are serialised alongside their respective models so that the same category-to-integer mappings are used at inference time.

Log transformation is applied to price targets using `numpy.log1p` before training. The choice of `log1p` over the natural logarithm avoids undefined values for prices of zero and provides numerically stable behaviour for small positive prices. At inference time, predictions are back-transformed using `numpy.expml` before being returned in the response.

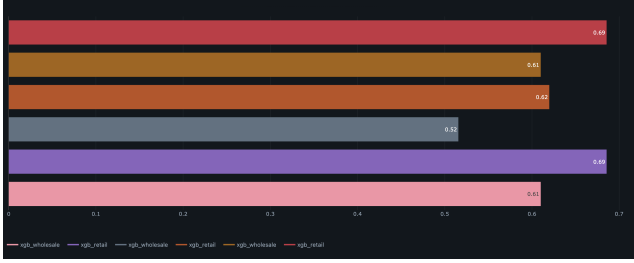


Fig. 3. R^2 values for the chained inference runs

C. Model Artifacts

Trained model artifacts are serialised to disk using `joblib` as four files: `model_wholesale.joblib`, `model_retail.joblib`, `encoder_wholesale.joblib`, and `encoder_retail.joblib`. These files are excluded from version control via a `.gitignore` rule. Storing large binary artifacts in a source repository inflates clone sizes, makes diffs meaningless, and conflates model versioning with code versioning [9]. The artifacts are expected to be present in the application working directory and are loaded by the prediction service on startup using `joblib.load`.

D. Reproducibility Measures

Reproducibility is addressed at four levels. Fixed random seeds are set during model training via the `random_state` parameter to ensure that results can be recreated from the same training data. All Python dependencies are pinned to minimum version bounds in `requirements.txt`, which is used consistently by both the CI environment and the Docker container to eliminate environment-specific discrepancies. Separate encoder instances for each model stage are serialised and loaded together with their respective models, ensuring that the category mappings applied at inference are identical to those applied during training. In the CI pipeline, a `conftest.py` fixture trains minimal mock models from randomly generated data before the test suite runs, ensuring that the testing environment is self-contained and does not require access to the real trained artifacts.

V. EXPERIMENT TRACKING

A. Why Experiment Tracking Matters

When training machine learning models it is easy to lose track of which settings produced which results. Without a tracking system, comparing two training runs requires manually checking notes or re-running experiments from scratch. Experiment tracking solves this by automatically recording everything about each training run: the settings used, the results achieved, and the model artifact produced. This makes it straightforward to compare runs, reproduce a previous result, or roll back to an earlier model version if something goes wrong.

B. MLflow Tracking Setup

AgriPrice uses MLflow [10] as its experiment tracking tool. MLflow is an open-source platform that records model training runs in a structured way. The tracking server is configured to store data in a local SQLite database file (`mlflow.db`), which means no separate server infrastructure is required. Every training run is automatically saved with a timestamp, making the full history of all experiments queryable at any time.

The experiment is named `AgriPrice_Chained_XGB_Final` and contains two distinct runs, one for each model stage. This separation makes it easy to view the performance of the wholesale model and the retail model independently, and to retrain just one stage if needed without affecting the other.

C. What Gets Tracked

For each training run, MLflow records the following information. On the parameters side, it logs the number of trees used (`n_estimators = 1000`), the maximum depth of each tree (`max_depth = 10`), the learning rate (`learning_rate = 0.02`), the subsample fraction (`subsample = 0.8`), and the column sampling fraction (`colsample_bytree = 0.8`). On the metrics side, it logs the R^2 score and the Root Mean Squared Error (RMSE) for each model evaluated on the held-out test set.

D. Model Performance Results

Table II summarises the performance metrics logged in MLflow for both model stages. The wholesale model achieved an R^2 of 0.60, meaning it explains 60 percent of the variation in wholesale prices. The retail model achieved an R^2 of 0.68, meaning it explains 68 percent of the variation in retail prices. The stronger performance of the retail model reflects the benefit of having the wholesale estimate as an additional input feature, which gives the retail model a strong price signal to condition on.

TABLE II
MLFLOW TRACKED MODEL PERFORMANCE

Model Stage	R^2 Score	MLflow Run Name
Stage 1 (Wholesale)	0.60	Stage1_Wholesale_XGB
Stage 2 (Retail)	0.68	Stage2_Retail_XGB

These base metrics serve a dual purpose. During development, they tell us how well the models perform on historical data. After deployment, they become the reference baseline against which the monitoring pipeline checks whether model accuracy has changed over time.

E. Model Registration

After each training run, the trained model is registered in the MLflow Model Registry under a named entry: `AgriPrice_XGB_Wholesale` for the first stage and `AgriPrice_XGB_Retail` for the second. Model registration creates a versioned, named record in the MLflow database

that links back to the full training run. This means the exact model currently in production can always be traced back to the specific training run that produced it, including all the parameters and metrics that were logged at the time.

VI. DEPLOYMENT

A. FastAPI Prediction Service

The prediction service is implemented in `predict.py` using the FastAPI framework [7]. Two HTTP endpoints are exposed. A GET endpoint at the root path `/` returns an acknowledgement message confirming that the service is active, providing a health check target for infrastructure monitoring. A POST endpoint at `/predict` accepts a JSON request body conforming to the input schema and returns a JSON response conforming to the output schema (Fig 4).

Request validation is handled automatically by Pydantic. If a required field is absent or of the wrong type, FastAPI rejects the request with an HTTP 422 Unprocessable Entity response before the prediction logic is executed. If the date field is present but in an incorrect format, the prediction handler raises a structured error response with HTTP 200 and an `error` key rather than allowing an unhandled exception to propagate. Table III summarises the API contract.

TABLE III
AGRIPRICE API CONTRACT

Field	Direction	Description
commodity	Input	Crop name string
classification	Input	Crop type string
market	Input	Market name string
county	Input	County name string
unit	Input	Unit of measure string
date	Input	Date string DD/MM/YYYY
stage_1_wholesale_estimate	Output	Wholesale price (KES)
stage_2_retail_forecast	Output	Retail price (KES)
currency	Output	Always KES
input_summary	Output	Echoed request fields

B. Docker Containerisation

The prediction service is containerised using Docker [8] with a `python:3.12-slim` base image, chosen for its minimal footprint. The Dockerfile follows a two-stage copy pattern: `requirements.txt` is copied and dependencies are installed first to leverage Docker’s layer caching, and `predict.py` is copied afterwards. The container exposes port 8000 and is configured to launch the Uvicorn ASGI server as its entrypoint.

Model artifact files (`.joblib`) are intentionally excluded from the Docker image. The image contains only the service code and its Python dependencies. Model files must be mounted into the container’s working directory at runtime using a Docker volume mount. This separation allows the model artifacts to be updated independently of the service code without rebuilding the image, which is important in a production context where model retraining may occur on a different schedule from code deployment.

C. GitHub Container Registry

Built Docker images are pushed to the GitHub Container Registry (GHCR) under the repository owner’s namespace. The image is named `agriprice-predict-service` and tagged as `latest` on every successful deployment. GHCR requires image names to be entirely lowercase; the workflow enforces this using a shell transformation on the repository owner’s username before constructing the image reference. The registry serves as the distribution point from which the image can be pulled and run in any target environment with Docker installed.

D. Docker Compose Infrastructure

Beyond the prediction service itself, AgriPrice runs a supporting infrastructure stack defined in `docker-compose.yml`. This stack launches three services together with a single command. A PostgreSQL database stores live prediction logs and computed drift metrics over time. Adminer provides a web-based interface for querying the database directly using SQL, accessible on port 8080. Grafana provides a visual dashboard interface for monitoring trends and alerts, accessible on port 3000. All three services are configured to start automatically and depend on each other in the correct order, with the monitoring and visualisation services waiting for the database to be ready before starting.

VII. MONITORING PIPELINE

A. Why Monitoring is Necessary

A model that performs well when it is first trained may gradually become less accurate over time. This happens because the real world changes: prices shift due to new economic conditions, new markets open, seasonal patterns change, or new commodity types become popular. If nobody is watching, a model that has become inaccurate continues to serve wrong predictions without anyone knowing. Monitoring is the process of continuously checking whether the deployed model is still behaving the way it should, and raising an alert when something looks wrong.

AgriPrice implements a five-step monitoring pipeline, illustrated in Fig 5, that runs on a schedule against live prediction data.

B. Step 1: Reference Dataset

The monitoring pipeline begins with a reference dataset, which acts as the baseline for everything else. The reference dataset is taken from the KAMIS historical training data used to train the models. It captures what normal looks like: the typical distribution of commodity types, markets, counties, and price ranges that the models were trained on.

The script `prepare_reference.py` processes this data and computes the base performance metrics from the training period. These base metrics are stored and used later as the benchmark against which live performance is compared. The base R^2 scores recorded at training time are 0.60 for the wholesale model and 0.68 for the retail model. Any significant drop from these values during production triggers an alert.

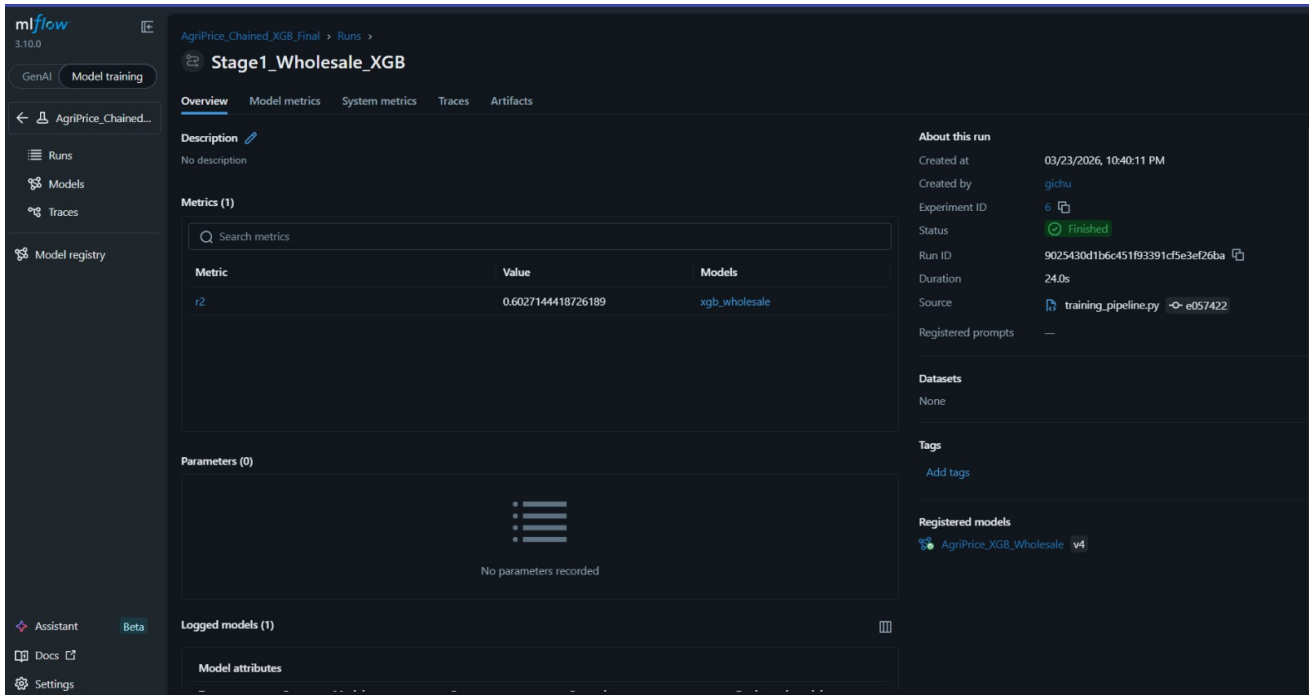


Fig. 4. MLflow UI screenshot showing the experiment runs Stage1_Wholesale_XGB with their logged R² scores

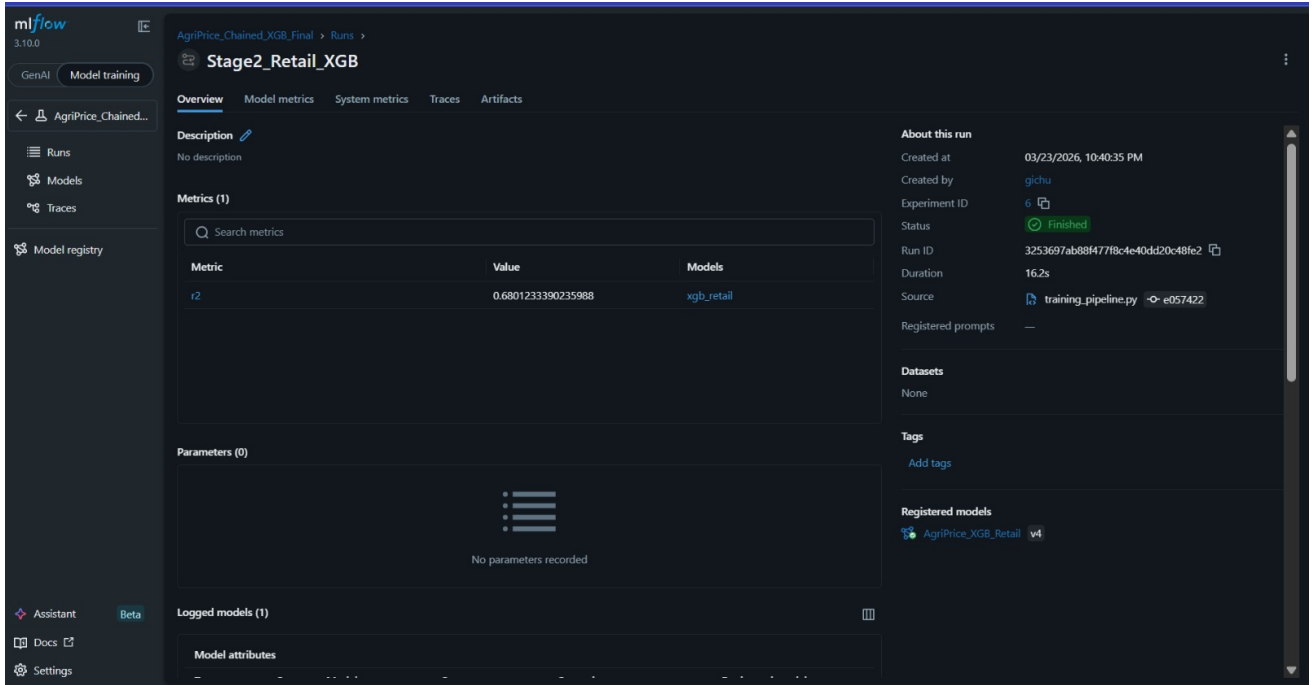


Fig. 5. MLflow UI screenshot showing the experiment runs Stage2_Wholesale_XGB with their logged R² scores

C. Step 2: Production Data and Predictions

Every time a user sends a request to the FastAPI prediction endpoint, the system logs the input features and the resulting predictions. The script `generate_batch.py` collects these live requests into batches for periodic evaluation. Each batch represents a window of recent production traffic, typically

covering a fixed number of requests or a time interval.

The production data passes through both model stages in exactly the same way as a normal prediction request. The wholesale model processes the input features first, producing $\hat{p}_w$. The retail model then receives the original features plus $\hat{p}_w$ and produces the final retail forecast $\hat{p}_r$. Both outputs are

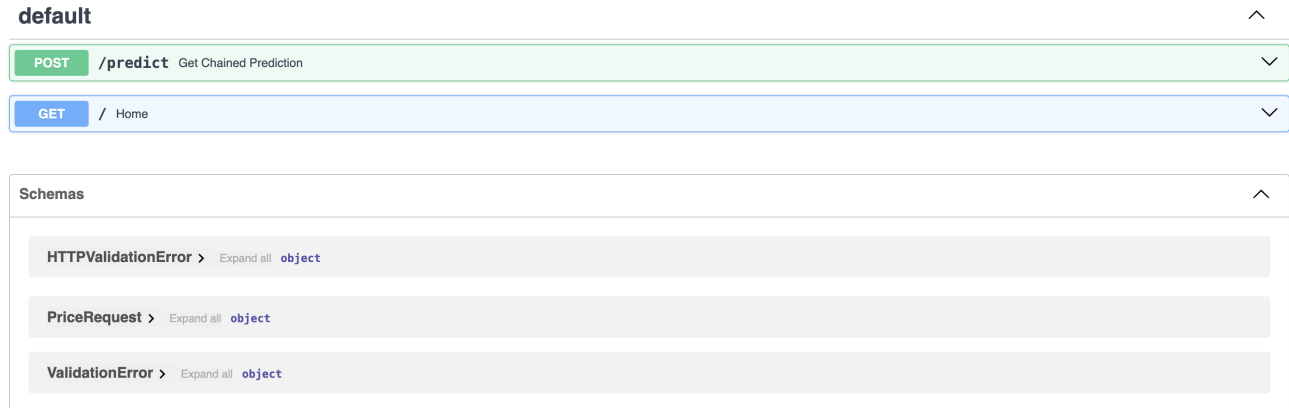


Fig. 6. FastAPI interactive documentation at /docs

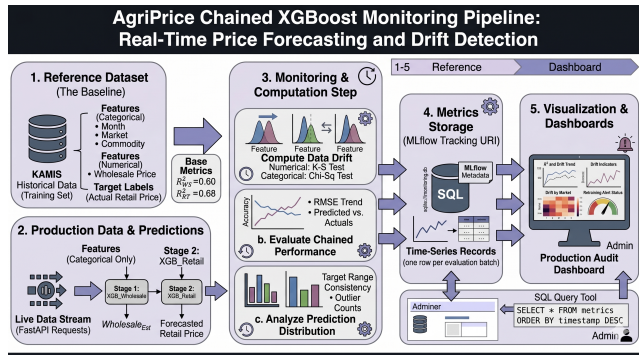


Fig. 7. AgriPrice monitoring pipeline: from reference baseline to live drift detection and dashboard alerts

recorded alongside the original input features for drift analysis.

D. Step 3: Monitoring and Computation

This is the core analytical step, implemented in `calculate_metrics.py`. It compares the production batch against the reference dataset across three dimensions.

Data drift detection checks whether the incoming feature values have shifted away from the training distribution. For numerical features such as wholesale price, the Kolmogorov-Smirnov (K-S) test is used. The K-S test measures how different two distributions are by looking at the largest gap between their cumulative distribution functions. A high K-S statistic means the production data looks very different from training data. For categorical features such as commodity, market, and county, the Chi-squared test is used instead. The Chi-squared test checks whether the frequency of each category in production matches what was seen during training. Both tests produce a p-value; a low p-value indicates statistically significant drift.

Chained performance evaluation measures how accurately the models are performing on recent data, where actual prices are known. It computes the RMSE trend over time, comparing predicted wholesale and retail prices against observed market prices. A rising RMSE over successive evaluation batches is a warning sign that model accuracy is degrading.

Prediction distribution analysis checks whether the outputs themselves look reasonable, independently of whether actual prices are available. It counts how many predictions fall outside the expected price range for each commodity and unit combination, flags unusual outlier counts, and checks whether the predicted wholesale and retail prices maintain a sensible relationship to each other.

E. Step 4: Metrics Storage

All computed metrics from Step 3 are stored in two places. MLflow logs the metrics under the experiment tracking URI `sqlite:///monitoring.db`, creating a time-stamped record for each evaluation batch. A SQL database stores the same metrics as time-series records, one row per evaluation batch, which allows the Grafana dashboard to query and plot trends over time. The combination of MLflow and SQL storage means the full audit trail of monitoring results is always available for inspection, whether through the MLflow UI, the Adminer SQL interface, or the Grafana dashboard.

VIII. VISUALISATION AND DASHBOARDS

A. Overview

Raw monitoring metrics stored in a database are not easy for a person to interpret at a glance. Visualisation tools turn those numbers into charts and indicators that make it immediately obvious whether the system is healthy or whether something needs attention. AgriPrice uses two visualisation tools: Grafana for real-time dashboard monitoring and Adminer for direct database inspection.

B. Grafana Dashboard

Grafana is a widely used open-source visualisation platform that connects to database sources and renders live charts that update automatically. In the AgriPrice monitoring stack, Grafana connects to the PostgreSQL database where monitoring metrics are stored and displays them across four panels.

The **R² and Drift Trend** panel shows how model accuracy has changed over time. The R² scores for both the wholesale and retail models are plotted as lines over successive evaluation batches. A declining line tells the team that the models are becoming less accurate and that retraining should be considered.

The **Drift Indicators** panel shows the current drift status for each feature. Each feature is represented by a bar or indicator light that turns from green to red as the K-S or Chi-squared test statistic increases. This gives an immediate visual signal of which features are drifting most strongly.

The **Drift by Market** panel breaks down drift by geographic market. Because Kenyan commodity prices vary significantly by region, this panel makes it easy to spot whether drift is happening uniformly across all markets or whether it is concentrated in specific locations. Concentrated drift in one market might indicate a local supply shock rather than a general model failure.

The **Retraining Alert Status** panel displays a gauge or indicator showing the current overall health of the system. When the combination of accuracy drop and drift scores crosses a defined threshold, this panel turns red and signals that the models should be retrained on updated data.

C. Adminer Database Interface

Adminer is a lightweight web-based SQL client that provides direct access to the PostgreSQL database. While Grafana is designed for visual monitoring, Adminer is designed for inspection and debugging. A team member can open Adminer at port 8080, log in with the database credentials, and run any SQL query directly against the monitoring tables.

A typical query used during monitoring review retrieves the most recent evaluation batches in reverse chronological order:

```
SELECT * FROM metrics
ORDER BY timestamp DESC;
```

This returns a table showing every evaluation batch, including the timestamp, the computed drift scores for each feature, the RMSE values for both model stages, the number of outlier predictions flagged, and whether the retraining alert threshold was crossed. This raw view is useful when investigating a specific alert or verifying that new data has been correctly logged.

D. Starting the Monitoring Stack

The entire monitoring infrastructure, including PostgreSQL, Adminer, and Grafana, is launched with a single Docker Compose command:

```
docker-compose up -d
```

This starts all three containers in the background. Adminer becomes available at <http://localhost:8080> and Grafana at <http://localhost:3000>. The `-d` flag means the containers run in detached mode, so the terminal is free for other work. To stop the entire stack, the command `docker-compose down` shuts everything down cleanly.

IX. CI/CD AND AUTOMATION

A. Overview

The CI/CD pipeline is defined in a single GitHub Actions workflow file located at `.github/workflows/main.yml`. The workflow is triggered automatically on every push to the main branch and on every pull request targeting main. It consists of two jobs: Continuous Integration and Continuous Delivery, that execute in sequence with an enforced dependency between them.

The overall flow is illustrated schematically as follows. A developer pushes code to the main branch. GitHub Actions provisions a clean Ubuntu virtual machine and executes the CI job. If all seventeen tests pass, control passes to the CD job, which builds a Docker image and pushes it to GHCR. If any test fails, the CD job does not start.

B. Continuous Integration Job

The CI job runs on an `ubuntu-latest` virtual machine provisioned by GitHub. It performs four steps sequentially. The repository is checked out using `actions/checkout@v4`. Python 3.12 is configured using `actions/setup-python@v5`. All project dependencies are installed from `requirements.txt` using `pip`. The full test suite is executed using `pytest` with no arguments, relying on `pytest.ini` to discover all tests in the `tests/` directory.

The quality gate mechanism relies on the exit code contract of the `pytest` process. A passing test suite produces exit code 0; any test failure produces a non-zero exit code. GitHub Actions interprets a non-zero exit code as a step failure, halts the CI job, and marks the overall workflow run as failed. No further steps in the CI job or in the CD job are executed.

C. Continuous Delivery Job

The CD job is declared with a `needs: ci` configuration directive, which creates a hard dependency on the successful completion of the CI job. Additionally, a conditional expression `github.event_name == 'push' && github.ref == 'refs/heads/main'` restricts the CD job to push events on the main branch only, preventing Docker image publication from pull request runs.

The CD job performs three substantive steps. It logs into GHCR using the `docker/login-action@v3` action with the repository's automatically provisioned `GITHUB_TOKEN` secret, which requires no manual credential configuration. It builds the Docker image using the project Dockerfile, applying a lowercase-transformed image name to satisfy GHCR's naming requirements. It pushes the built image to the registry.

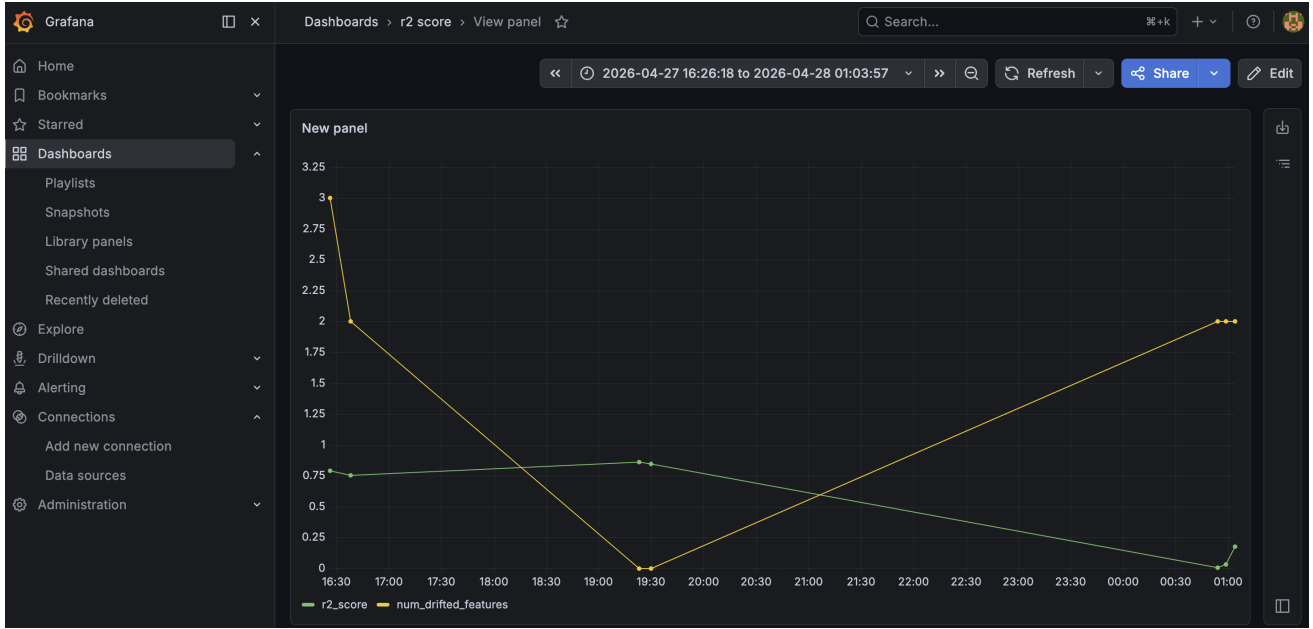


Fig. 8. Grafana dashboard screenshot showing the R^2 trend panel, drift indicators, drift by market, and retraining alert status panels

Language: English PostgreSQL db agri_monitoring public Select: metrics postgres

Adminer 5.4.2

DB: agri_monitoring Schema: public

SQL command Import Export Create table

Select metrics

Select data Show structure Alter table New item

Select Search Sort Limit 50 Text length 100 Action Select

SELECT * FROM "metrics" LIMIT 50 (0.003 s) Edit

	timestamp	batch_id	batch_size	num_drifted_features	share_drifted_features	rmse	r2_score
edit	2026-04-21 08:58:04.588787	batch_1776761802	500	2	0.5	262.421454544402	0.0330787213163761
edit	2026-04-27 14:36:55.480311	batch_1777300595	500	1	0.25	117.80469942154	0.933510296062574
edit	2026-04-27 20:27:54.122315	batch_1777321641	500	2	0.5	19.8467952729114	0.978769391642154
edit	2026-04-28 00:55:52.794011	batch_1777337709	500	1	0.25	55.8818477131195	0.79468301196279
edit	2026-04-28 00:57:37.416814	batch_1777337849	500	2	0.5	81.7663085277217	0.698950234296022
edit	2026-04-28 00:58:03.718523	batch_1777337874	500	1	0.25	52.2154178230941	0.940934172206971

Whole result 6 rows Modify Selected (0) Export (6)

Save Edit Clone Delete

Import

Fig. 9. Adminer at <http://localhost:8080>

D. Pipeline Integrity

The sequential structure of CI before CD is a deliberate architectural decision that reflects a core MLOps principle: the container registry should be a trusted repository of deployable artifacts. Allowing images to be published without passing automated tests on a clean environment would undermine this guarantee. The clean Ubuntu environment in which tests run is critical because it eliminates the “works on my machine” failure mode: if a dependency is missing from `requirements.txt`, the CI machine will fail where a developer’s pre-configured local environment would succeed.

Table IV summarises the two pipeline jobs and their key properties.

TABLE IV
CI/CD PIPELINE JOB SUMMARY

Property	CI Job	CD Job
Trigger	Push / PR to main	Push to main only
Depends on	–	CI job (must pass)
Environment	ubuntu-latest	ubuntu-latest
Python version	3.12	–
Key action	Run 17 pytest tests	Build + push Docker
Failure effect	Blocks CD	Workflow fails
Registry write	No	Yes (GHCR)

X. TESTING STRATEGY

A. Test Suite Overview

The project test suite comprises seventeen tests distributed across two files. `test_prediction.py` contains nine tests that verify model inference logic and related infrastructure.

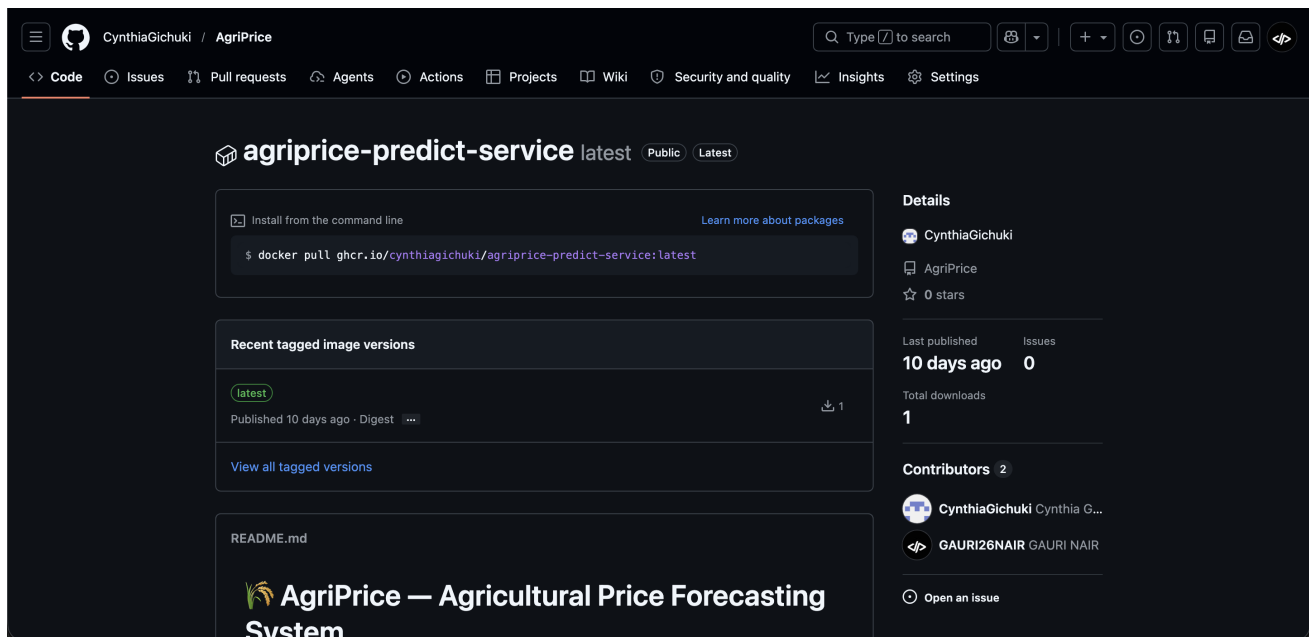


Fig. 10. GitHub Actions CI/CD pipeline showing successful completion of both the CI testing job and the CD Docker deployment job

```
gichu@cynthia MINGW64 ~/Documents/MSLU/Semester 2/Artificial Intelligence/AgriPrice (main)
$ pytest -v
===== test session starts =====
platform win32 -- Python 3.11.7, pytest-7.4.0, pluggy-1.0.0 -- C:\Users\gichu\anaconda3\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\gichu\Documents\MSLU\Semester 2\Artificial Intelligence\AgriPrice
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.9.0
collected 17 items

tests/test_api.py::test_home_endpoint_is_alive PASSED [ 5%]
tests/test_api.py::test_predict_valid_request_returns_200 PASSED [ 11%]
tests/test_api.py::test_predict_response_has_required_keys PASSED [ 17%]
tests/test_api.py::test_predict_currency_is_kes PASSED [ 23%]
tests/test_api.py::test_predict_prices_are_positive PASSED [ 29%]
tests/test_api.py::test_predict_input_summary_echoes_request PASSED [ 35%]
tests/test_api.py::test_predict_invalid_data_returns_error_message PASSED [ 41%]
tests/test_api.py::test_predict_missing_field_returns_422 PASSED [ 47%]
tests/test_prediction.py::test_chained_inference_flow PASSED [ 52%]
tests/test_prediction.py::test_wholesale_wrong_feature_count_raises_error PASSED [ 58%]
tests/test_prediction.py::test_retail_missing_wholesale_feature_raises_error PASSED [ 64%]
tests/test_prediction.py::test_prediction_output_is_float PASSED [ 70%]
tests/test_prediction.py::test_predictions_within_realistic_kes_range PASSED [ 76%]
tests/test_prediction.py::test_valid_date_format_parses_correctly PASSED [ 82%]
tests/test_prediction.py::test_wrong_date_format_raises_error PASSED [ 88%]
tests/test_prediction.py::test_garbage_date_raises_error PASSED [ 94%]
tests/test_prediction.py::test_data_folder_exists PASSED [100%]

===== 17 passed in 2.12s =====
```

Fig. 11. Test sets used

`test_api.py` contains eight tests that verify the HTTP API contract. All tests are discovered and executed by `pytest`, which is configured via `pytest.ini` to search the `tests/` directory and to include the project root on the Python module path using the `pythonpath = .` setting (Fig 5).

B. Model Logic Tests

The nine tests in `test_prediction.py` target the core inference behaviour of the chained model system. The test for chained inference flow verifies that the wholesale model produces a positive output and that this output can be passed as a feature to the retail model to produce a second positive output, confirming the end-to-end pipeline operates correctly. Two tests verify that the models enforce their expected feature dimensionality by raising exceptions when inputs with incorrect feature counts are supplied. A test for output type checks that the final prediction is a Python `float` rather than a `numpy` scalar type, which is important for JSON

serialisation. A test for price range verifies that predictions fall within a realistic positive KES range. Three tests exercise date parsing logic, confirming that the expected `DD/MM/YYYY` format succeeds and that `ISO` format and garbage strings fail with appropriate exceptions. A final infrastructure test verifies that the `data/` directory is present in the project structure.

C. API Contract Tests

The eight tests in `test_api.py` target the HTTP layer of the prediction service using FastAPI's `TestClient`, which is backed by the `httpx` library and requires no running server process. Tests verify that the home endpoint returns HTTP 200, that a well-formed prediction request returns HTTP 200, that the response JSON contains all four required keys, that the currency field always contains the string `KES`, that both price fields are greater than zero, that the input summary echoes the commodity, market, and date sent in the request, that an incorrectly formatted date returns a graceful error message rather than a server crash, and that a request body missing required fields returns HTTP 422.

D. Mock Model Strategy

A fundamental challenge in CI testing for ML systems is that the real trained model artifacts are excluded from the repository. The `conftest.py` file addresses this with a session-scoped, autouse `pytest` fixture named `create_test_artifacts`. Before the test session begins, this fixture trains four minimal XGBoost models and `OrdinalEncoder` instances from two hundred randomly generated data points with the same feature schema as the real models. The fixture uses `autouse=True` so that it runs automatically for every test without requiring each test to declare a dependency.

After all tests complete, the fixture removes the generated artifact files. The `api_client` fixture imports `predict.py` inside the fixture function body rather than at module level, deferring the import until after the mock artifacts exist and the `joblib.load` calls in `predict.py` will succeed.

Table V provides a complete summary of all seventeen tests.

TABLE V
COMPLETE TEST SUITE SUMMARY

Test Name	File	What It Verifies
Chained inference flow	pred.py	Wholesale output feeds into retail
Wrong feature count	pred.py	Error on wrong input dimensions
Missing wholesale feat.	pred.py	Retail model rejects 9-feature input
Output is float type	pred.py	Prediction is Python float
Prices within KES range	pred.py	Output is a positive realistic value
Valid date parses	pred.py	DD/MM/YYYY format accepted
Wrong date format fails	pred.py	ISO format raises an error
Garbage date fails	pred.py	Non-date string raises an error
Data folder exists	pred.py	data/ directory is present
Home endpoint alive	api.py	GET / returns HTTP 200
Valid request 200	api.py	POST /predict returns HTTP 200
Required keys present	api.py	Response has all 4 JSON fields
Currency is KES	api.py	Currency field equals KES
Prices are positive	api.py	Both price fields are > 0
Input summary echoed	api.py	Response reflects request fields
Invalid date error	api.py	Wrong date returns error message
Missing field 422	api.py	Incomplete body returns HTTP 422

XI. DISCUSSION AND LIMITATIONS

A. Runtime Model Mounting

The decision not to bake model artifacts into the Docker image introduces an operational dependency: the image cannot produce predictions unless the correct model files are mounted into the container at runtime using a volume mount. While this keeps the image lean and decouples model versioning from service versioning, it requires additional operational steps to provision artifacts in deployment environments. A more complete solution would integrate a dedicated model registry such as MLflow Model Registry [10] or DVC to manage artifact versioning, lineage, and retrieval automatically, eliminating the manual mounting step.

B. Current Monitoring Coverage and Future Improvements

The monitoring pipeline described in Section VII provides meaningful coverage of data drift and model performance over time. However, there are areas where it could be strengthened. The current drift detection relies on batch evaluation rather than continuous real-time monitoring, which means a sudden spike in bad predictions within a single batch might not be caught immediately. A future improvement would be to

set up streaming monitoring that evaluates each prediction individually as it arrives, rather than waiting for a batch to accumulate. Additionally, the accuracy monitoring component requires actual observed market prices to compare against predictions. Currently this depends on manually sourcing recent price data from public databases such as KNBS or FAO. Automating this data retrieval would make the accuracy monitoring fully self-contained.

C. CI Fidelity of Mock Models

The mock models used during CI are trained on uniformly random data and therefore do not reflect the behaviour of the real trained models in any meaningful sense. Tests that check prediction values, such as the price range test and the price positivity test, verify only that the mock models produce positive outputs for the given inputs, not that the production models produce accurate or calibrated predictions. This is a known limitation of the mock model approach: it validates code paths and API contracts efficiently on a clean machine without requiring access to the real artifacts, but it cannot serve as a regression test for model quality. A separate integration test stage with access to the real artifacts would be required to detect regressions in prediction accuracy.

D. Single-Stage Deployment

The current CD pipeline pushes a single `latest` tag to GHCR on every successful deployment. This means that rolling back to a previous version requires rebuilding the image from a prior commit rather than simply re-deploying a tagged artifact. A more robust versioning strategy would tag images with the Git commit SHA or a semantic version number in addition to `latest`, creating an immutable audit trail of deployable artifacts that can be re-deployed without rebuilding.

E. Alignment with Sustainable Development Goals

AgriPrice contributes indirectly to the United Nations Sustainable Development Goal 2 (Zero Hunger) [11] by improving price transparency in Kenyan agricultural markets. Farmers who have access to reliable price forecasts can make more informed decisions about when and where to sell their produce, reducing the information asymmetry that currently disadvantages smallholder farmers in negotiations with traders and intermediaries. Improved price predictability can also reduce post-harvest losses by enabling farmers to time sales more effectively. While AgriPrice does not directly address food production or nutrition, the economic empowerment of smallholder farmers through better market information is a recognised pathway to improved food security outcomes.

XII. CONCLUSION

This paper presented AgriPrice, an end-to-end MLOps pipeline for Kenyan agricultural commodity price forecasting. The system combines a chained XGBoost modelling architecture, in which a wholesale price model feeds its output into a retail price model, with a FastAPI prediction service,

Docker containerisation, MLflow experiment tracking, a drift-aware monitoring pipeline, Grafana and Adminer visualisation dashboards, and an automated GitHub Actions CI/CD pipeline.

Seventeen automated tests covering model inference logic and HTTP API contract compliance are executed on a clean Ubuntu environment before any deployment occurs. This provides a reliable quality gate that prevents untested code from reaching the GitHub Container Registry. The use of a session-scoped `conftest.py` fixture that trains minimal mock models from random data resolves the challenge of testing a model-dependent service in an environment where the real trained artifacts are unavailable.

The project covers all six components required by the ARTIFIN course project specification: a publicly available dataset was selected (KAMIS Kenyan market data), a clear ML goal was defined (chained wholesale and retail price regression), models were trained with full experiment tracking using MLflow, the models were deployed in production via FastAPI and Docker, performance is monitored over time using drift detection and accuracy evaluation, and the entire serving pipeline is automated through CI/CD with GitHub Actions.

The primary areas for future improvement are automating the retrieval of actual market prices for accuracy monitoring, adding real-time streaming monitoring alongside the current batch approach, and integrating a model registry to manage artifact versioning and eliminate the manual mount step at deployment time.

The broader lesson of the AgriPrice project is that model development and model deployment are equally demanding engineering disciplines. A model that performs well in a notebook but cannot be reliably shipped, tested, monitored, and maintained delivers limited real-world value. The MLOps practices demonstrated here provide the foundation for an agricultural price forecasting system that can be trusted, updated, and scaled in a production context.

REFERENCES

- [1] World Bank, “Kenya overview,” <https://www.worldbank.org/en/country/kenya/overview>, 2024, accessed: 2026-05-01.
- [2] D. Kreuzberger, N. Köhl, and S. Hirschl, “Machine learning operations (MLOps): Overview, definition, and architecture,” *IEEE Access*, vol. 11, pp. 31 866–31 879, 2023.
- [3] M. A. Hamjah, “Time-series forecasting of agricultural commodity prices using ARIMA models,” in *Journal of the Bangladesh Agricultural University*, 2014, vol. 12, no. 1, pp. 175–184.
- [4] A. Darekar and A. A. Reddy, “Cotton price forecasting using fuzzy ARIMA model,” *Journal of Cotton Research and Development*, vol. 31, no. 1, pp. cotton–pages, 2017.
- [5] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 785–794.
- [6] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, “Hidden technical debt in machine learning systems,” *Advances in Neural Information Processing Systems*, vol. 28, 2015.
- [7] S. Ramírez, “FastAPI documentation,” <https://fastapi.tiangolo.com>, 2024, accessed: 2026-05-01.
- [8] Docker Inc., “Docker documentation,” <https://docs.docker.com>, 2024, accessed: 2026-05-01.
- [9] N. Gift and A. Deza, *Practical MLOps*. O’Reilly Media, 2021.

- [10] Databricks Inc., “MLflow documentation,” <https://mlflow.org/docs/latest/index.html>, 2024, accessed: 2026-05-01.
- [11] United Nations, “Goal 2: Zero hunger,” <https://sdgs.un.org/goals/goal2>, 2023, accessed: 2026-05-01.